SMART EXPENSE SPLITTER ABSTRACT

ADVANCE DATA STRUCTURES MINI-PROJECT

NAME: Krishna Shree B - 2024115045

Nila PR - 2024115095

Samyukta Kamalakannan - 2024115007

CLASS: IT K BATCH

SEMESTER: 4

Introduction

- We created a web application that helps to manage shared expenses, which is a common challenge in everyday situations such as hostel living, group trips, or team activities.
- This application helps to track expenses, identify who paid, calculate how much each person owes, and determines how to settle debts fairly.
- Traditional methods often rely on manual calculations, which are prone to human error and inefficiency.
- Manually tracking money spent and settlement pathways often leads to an non-optimal number of transactions.
- The Smart Expense Splitter addresses these challenges by providing a simple, efficient, and automated solution for split management.
- Users can add members, record expenses, and split costs using various methods such as equal distribution, share-based allocation, and percentage-based splitting.
- The system generates an optimized settlement plan that significantly minimizes the number of transactions.

Problem Statement

- In group expense scenarios, individuals face several difficulties in tracking and settling shared costs.
- The lack of a structured system often leads to confusion, miscalculations, and unnecessary financial disputes.
- As the number of transactions grows, determining the minimum number of payments required to settle debts becomes increasingly complex.
- Existing solutions may lack the flexibility of diverse splitting methods or efficient settlement strategies.
- There is a need for a system that can accurately record expenses, support multiple splitting techniques, and compute an optimized settlement plan that ensures fairness while reducing transaction frequency.

System Architecture

- The Frontend Layer: Developed using HTML, CSS, and JavaScript, it provides an interactive UI for adding members, inputting expenses, selecting split methods, and viewing results.
- The Backend Layer: Implemented using Python with the Flask framework, it acts as a bridge between the user interface and the computational logic, processing requests and handling expense data.
- The Computational Module: Written in C++, it performs core logic using graph-based modeling to represent debts and applies priority queue algorithms to minimize transactions.
- Data Management: A Trie data structure is implemented in the C++ module to enable fast, efficient prefix-based searching of user names.

Modules Used

1. User Management Module

This module handles the addition and management of users in the system. Users can be dynamically added through the interface, and their names are stored for further processing. It ensures that duplicate entries are avoided and maintains the list of active participants involved in expense sharing.

2. Expense Management Module

This module is responsible for recording expenses. It captures details such as the amount spent, the payer, and the participants involved in the expense. The system supports multiple expense entries and maintains a structured representation of transactions using a graph-like model where each user's contribution is tracked.

3. Expense Splitting Module

This module provides flexibility in splitting expenses using different methods:

- Equal Split – divides the expense equally among selected users
- Share-Based Split – distributes the expense based on assigned shares
- Percentage-Based Split – allocates costs according to user-defined percentages

This module ensures accurate calculation of individual contributions based on the selected method.

4. Settlement Optimization Module

This is the core computational module implemented in C++. It processes all recorded transactions and computes the final balance for each user. Using efficient algorithms and priority queues, it minimizes the number of transactions required to settle all debts by matching creditors and debtors optimally.

5. Search Module

This module uses a Trie data structure to enable efficient prefix-based searching of user names. It improves usability by allowing quick filtering and selection of users while entering data.

Algorithm

Step 1: Graph Representation

All expenses are modeled using a directed graph:

- Each node represents a user
- A directed edge from user A to user B indicates that A owes money to B
- The edge weight represents the amount owed

Step 2: Net Balance Calculation

For each user, a net balance is computed:

- Balance > 0 → User is a creditor (should receive money)
- Balance < 0 → User is a debtor (should pay money)
- Balance = 0 → User is settled (no dues)

This step converts the graph into a simplified balance sheet.

Step 3: Priority Queue Initialization

Two priority queues (heaps) are used to optimize settlement:

- Max Heap (Creditors):
  Stores users with positive balances
  → Highest creditor is processed first
- Min Heap (Debtors):
  Stores users with negative balances
  → Highest debtor (most negative) is processed first

Step 4: Debt Settlement Process

The algorithm settles debts greedily:

- Extract the highest creditor and highest debtor
- Calculate the settlement amount:
  → `min(creditor balance, |debtor balance|)`
- Record the transaction
- Update both users' balances
- If any balance remains (non-zero), reinsert into the respective heap

Step 5: Repeat Until Settled

- Continue the process until both heaps are empty
- This ensures:
  - All debts are cleared
  - The minimum number of transactions is achieved

Step 6: Trie-Based Search Algorithm

For efficient user search functionality:

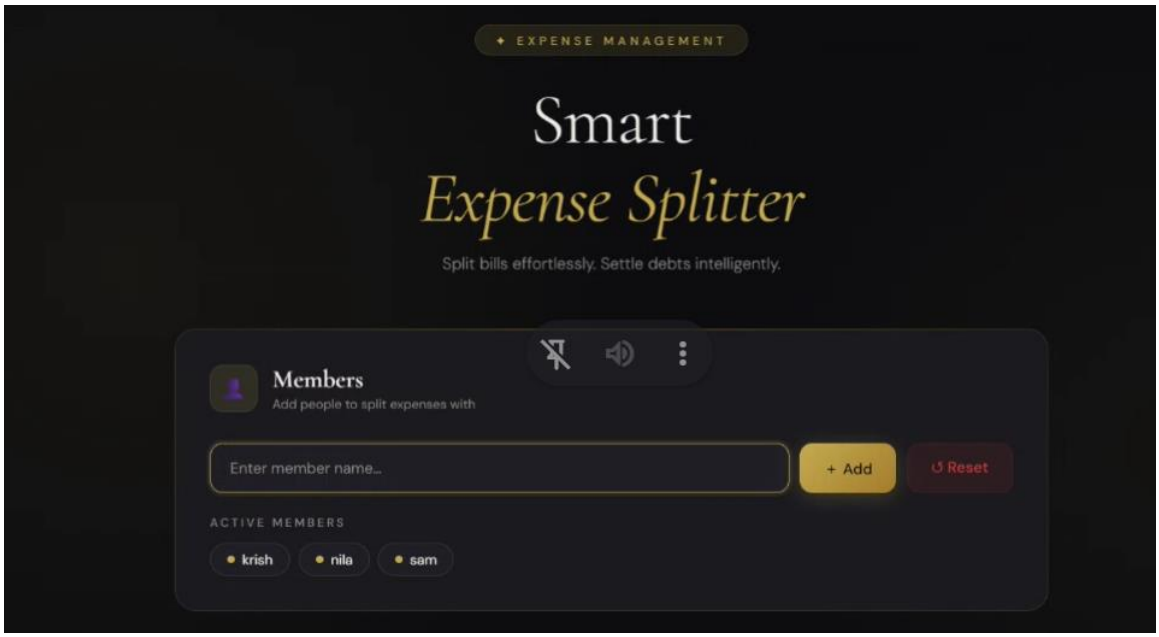
- Insert all usernames into a Trie (Prefix Tree)
- Traverse the Trie using the given prefix
- Perform Depth First Search (DFS) from that point
- Collect all matching usernames

This allows:

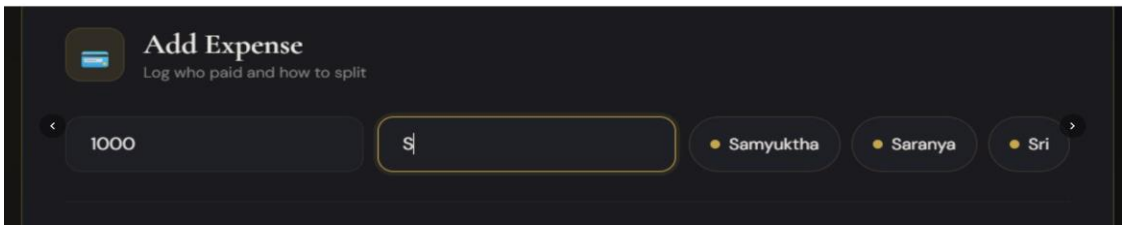
- Fast lookup
- Efficient prefix-based searching

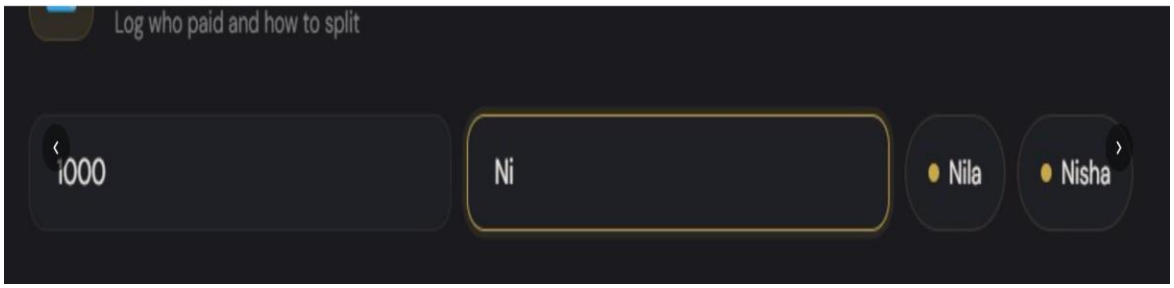
PROJECT OUTPUT

ADD USERS:

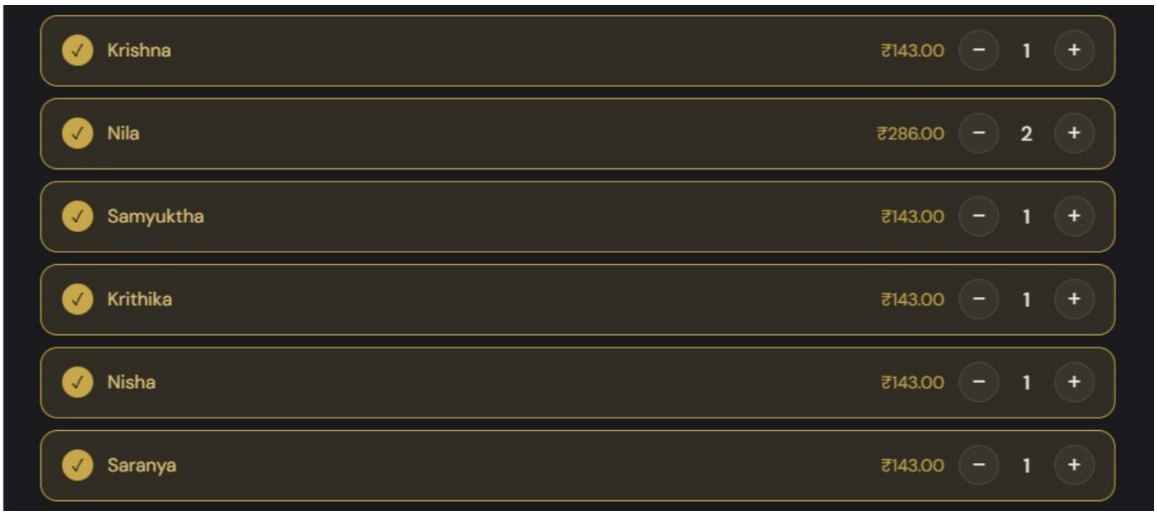


PREFIX SEARCH:

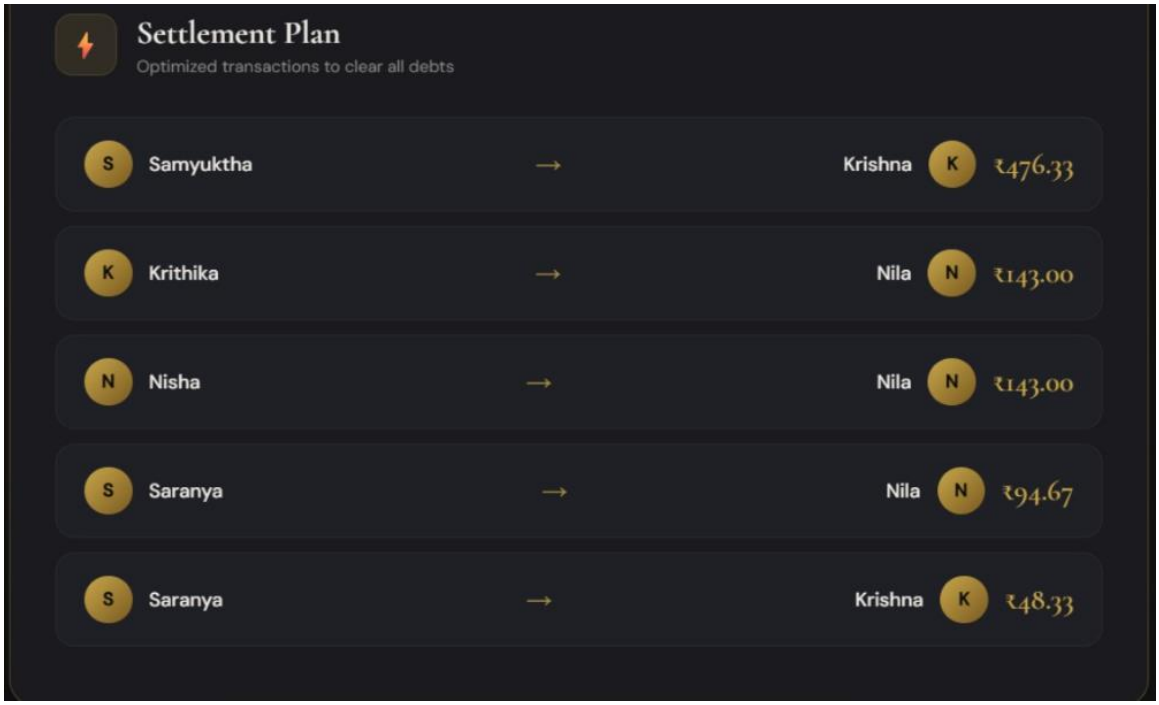




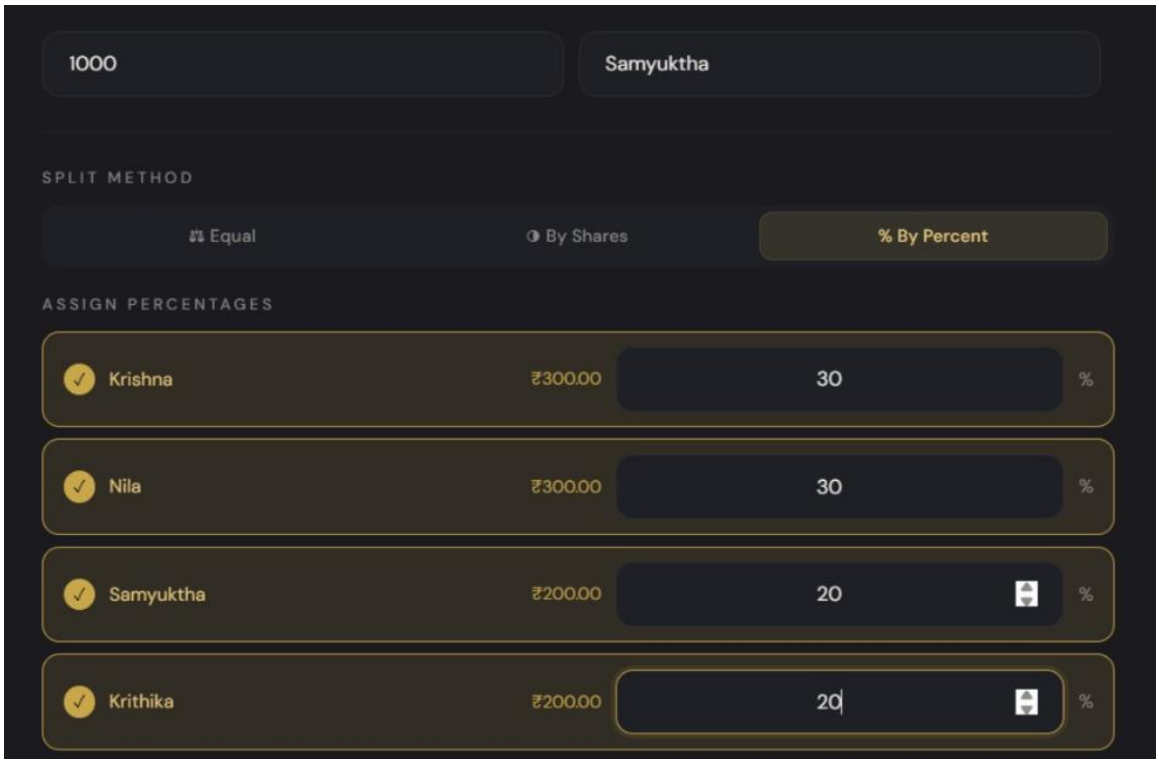
SPLIT:



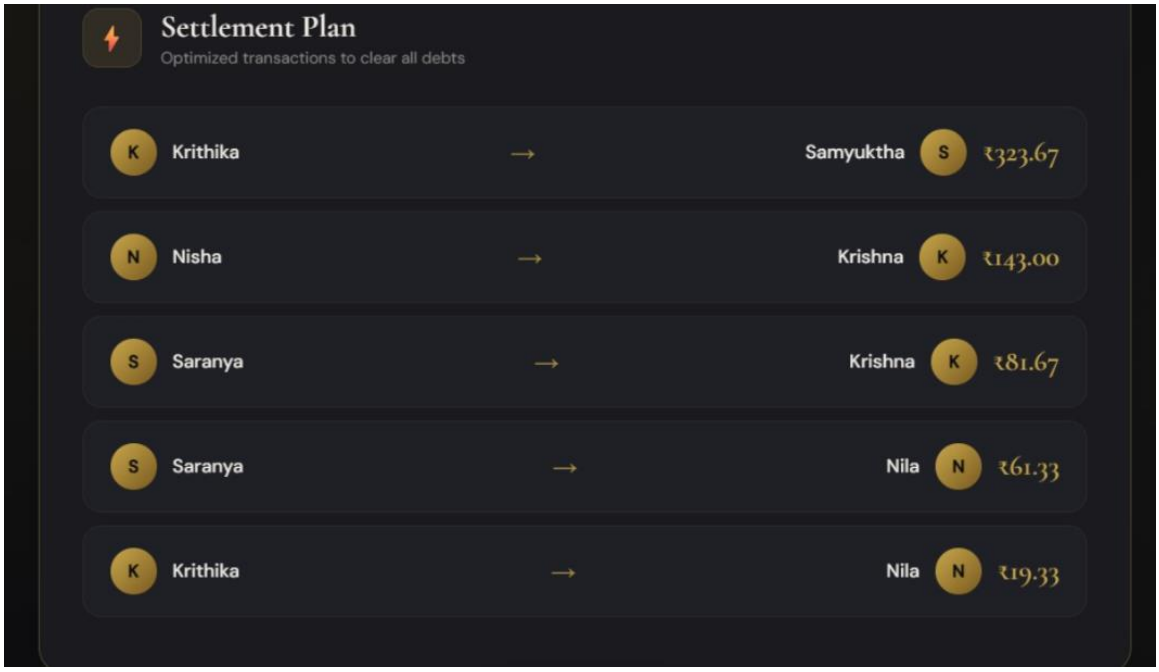
SPLIT EXPENSE BY SHARE:



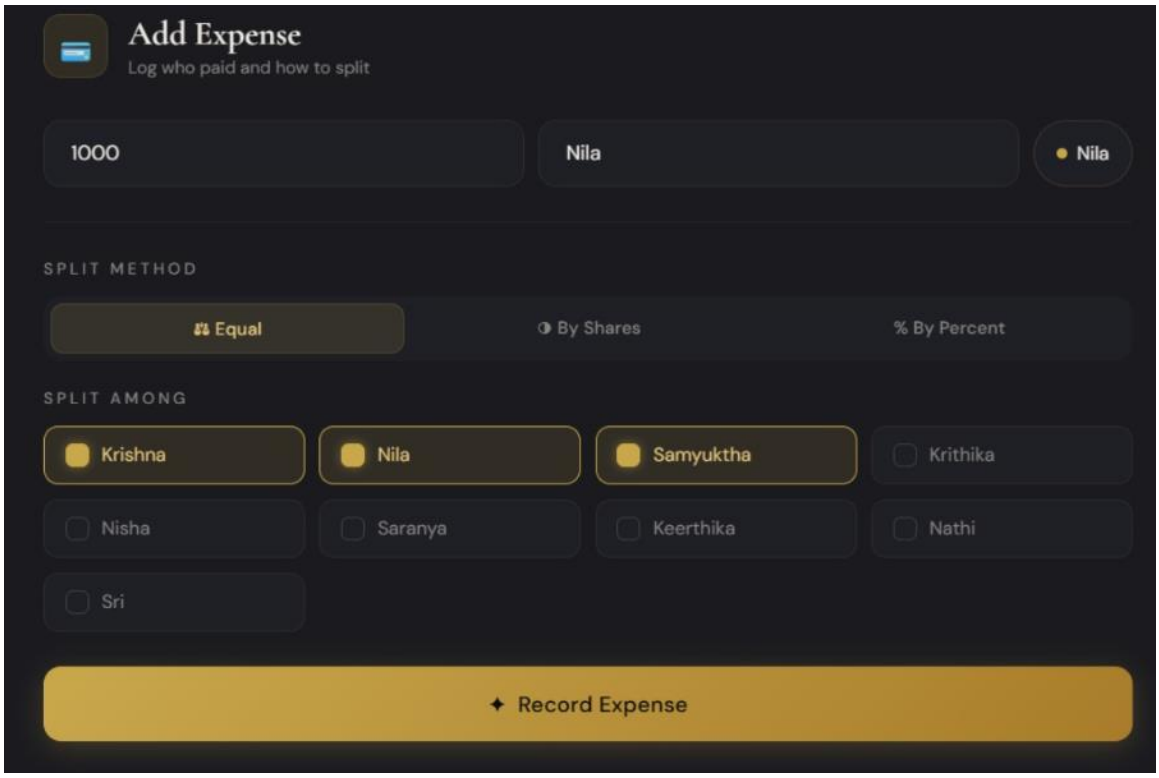
SPLIT BY PERCENTAGE:



OUTPUT AFTER SPLIT BY PERCENTAGE:



EQUAL SPLIT:



OUTPUT AFTER EQUAL SPLIT: